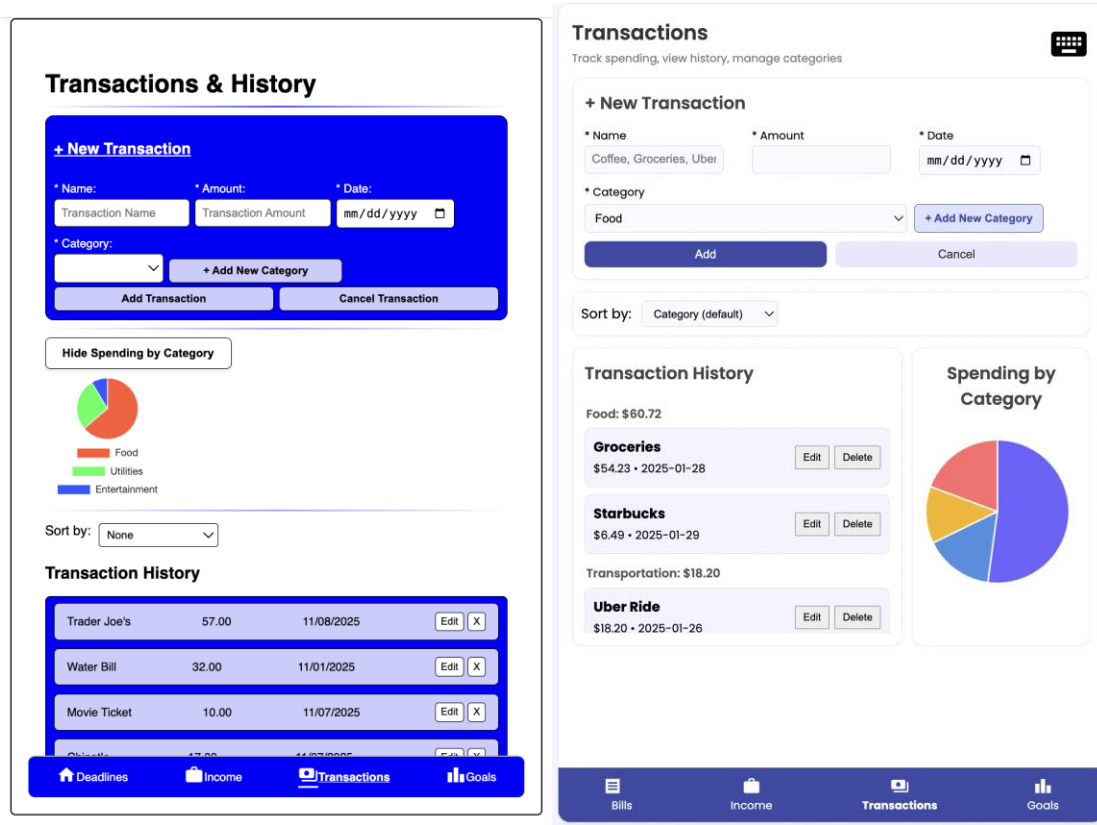


CMSC 434: Phase 4

Part A) Redesign

Transactions Page



The earlier version had an overwhelming color scheme, with a harsh contrast between the dark blue accents on the white background. In the new version, the color scheme uses lighter colors for a more visually appealing look.

Additionally, users gave feedback about poor space utilization on the earlier version, where users had to scroll on the screen to see all of the elements. The new version has an optimized layout where all main sections (+ New Transaction, Transaction History, and Spending by Category) are clearly visible without scrolling. The "+ New Transaction" section is still, unlike the earlier version which would expand when clicked.

Lastly, we added a keyboard placeholder (an element we missed on the earlier version) on the top right corner.

Goals Page

Savings & Spending

Savings100.00

GoalsTrack progress per goal

Goal name (e.g., Vacation)TargetAdd

Bali Vacation100.00 of 2500.00 (4%)

View Deposits (1)Record depositRecord

View Spending →

DeadlinesIncomeTransactionsGoals

Goals

Track savings goals and spending budgets

SavingsSpending

Total Saved\$100.00Across all savings goals

+ Add New Goal

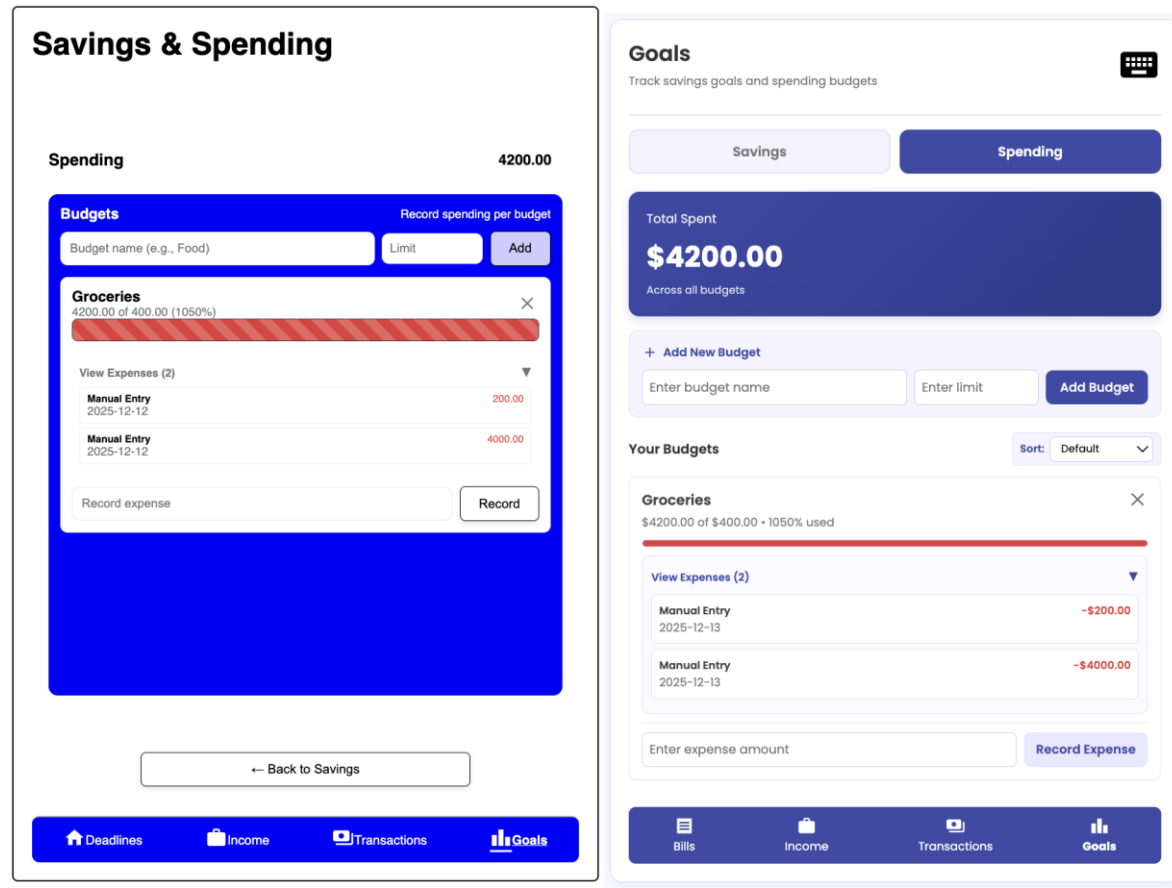
Enter goal nameEnter targetAdd Goal

Your GoalsSort: Default

Bali Vacation\$100.00 of \$2500.00 • 4% complete

View Deposits (1)Enter deposit amountRecord Deposit

BillsIncomeTransactionsGoals



There are two sets of screenshot pairs here to reflect changes made to both the “Savings” and “Spending” tabs of the “Goals” page.

To reduce unnecessary jargon and maintain consistency, we changed the page header from “Savings & Spending” to just “Goals”. We also removed the “Goals” and “Budgets” subtitles on the “Savings” and “Spending” pages, respectively (which also helped to declutter the interface).

In terms of the flipping card functionality that was categorized to be “disruptive”, we decided to change that to two simple buttons at the top, where users can simply toggle between the two screens without any potentially disorienting animations.

Moreover, we decided to change the general structure of some of the elements to better utilize space and reduce visual overload. Instead of having the total savings/spending in a small corner on the right, we made it the main display at the top. The “Add New Goal/Budget” section was separated from the list of existing items to ensure the text wasn’t so concentrated in one area, and that the flow is easier to scan. We also added a sorting functionality to help the user better find what they’re looking for and reduce the need for scrolling.

Finally, we received a lot of feedback about how it was hard to discern which elements were text inputs, so we relabeled all of these types of fields with clearer prompts like “Enter here”. We predict that this will better prompt the user to the right action, and again reduce confusion and cognitive load overall.

Bills Page (Previously Deadlines)

The image shows two side-by-side mobile app screens. The left screen, titled 'Deadlines & To-Do', has a blue header with two tabs: 'Deadlines' (active) and 'To-Do List'. Below the tabs is a 'Deadlines' section with an 'Add Deadline' form. The form has fields for 'Title (e.g., Rent due)', 'mm/dd/yyyy', and 'Add notes (optional)'. Below the form is a button 'Add' and a status 'No deadlines yet'. The right screen, titled 'Bills', has a blue header with the title 'Bills' and a subtitle 'Track payments, due dates, and upcoming expenses'. Below the header is a summary section showing '\$1,370 Total This Month', '\$1,260 Paid', and '\$110 Remaining'. Below this is a '+ New Bill' form with fields for 'Name', 'Due Date', 'Amount', 'Category', 'Status', and 'Notes'. Below the form are buttons 'Add Bill' and 'Cancel'. Below the form is a 'Your Bills' section with a list of bills: 'Electricity' (Unpaid), 'Internet' (Paid), 'Rent' (Paid), and 'Netflix'. The bottom of both screens has a navigation bar with icons for 'Deadlines', 'Income', 'Transactions', and 'Goals'.

Deadlines & To-Do

Deadlines

Add Deadline Earliest first Add

Title (e.g., Rent due) mm/dd/yyyy

Add notes (optional)

No deadlines yet

Bills

Track payments, due dates, and upcoming expenses

\$1,370 **\$1,260** **\$110**
Total This Month Paid Remaining

+ New Bill

* Name * Due Date
Electricity, Rent, Internet... mm/dd/yyyy

* Amount Category
\$ Utilities

* Status Notes
Unpaid Optional

Add Bill Cancel

Your Bills

☐ **Electricity** Unpaid
\$95.20 • Due Dec 20, 2025 • Utilities

☒ **Internet** Paid
\$60.00 • Due Dec 18, 2025 • Utilities

☒ **Rent** Paid
\$1,200.00 • Due Dec 1, 2025 • Housing

☐ **Netflix**

Deadlines **Income** **Transactions** **Goals**

Bills **Income** **Transactions** **Goals**

Since “Deadlines and To-Do” were too generic to be core features of a budgeting app, we decided to redesign it into a “Bills” page so that it makes more sense in this context.

Bills have a deadline and paying one’s bills counts as a financial task, so the structure of this page has a lot of similarities to the old version. The user is able to manage their bills by adding a bill to the list and specifying the name, due date, amount due, category (utilities, housing, subscription, loan), and status (unpaid or paid). They can then indicate whether a bill on the list has been paid off or not by clicking on the checkbox. The total amount due, the paid amount, and the unpaid amount are calculated automatically and displayed at the top, distinguished by different colors as an immediate visual reminder for the user. Furthermore, bills that are paid will be reflected as a transaction on the “Transactions” page.

Users mentioned some internal consistency issues. In the new version, the terminology and formatting of this page resembles the Transactions page because both have a box for adding a new entry and a list. Words like “Name”, “Amount”, and “Category” are used in both, with an asterisk to indicate that the field is mandatory.

Income Page

Income
Total Income 3938.46

+ Add Income

View Ledger

Title / Name of Income
e.g., Campus Barista, Freelance, Company Name

Type of Income

- ☐ Hourly
- ☐ Lump sum (odd jobs / one-off)
- ☐ Annual salary

Save income Reset

Deadlines Income Transactions Goals

Income
Track paychecks, campus jobs, and side gigs in one place.

Monthly income summary

Total from ledger **\$1,108.75**

View income statement

Income timeline
Visual schedule of upcoming recurring paychecks + one-time income.

Next 30 days All Manage recurring income

2025-12-12 → 2026-01-11 Upcoming total: \$1,241.00

+ Add income

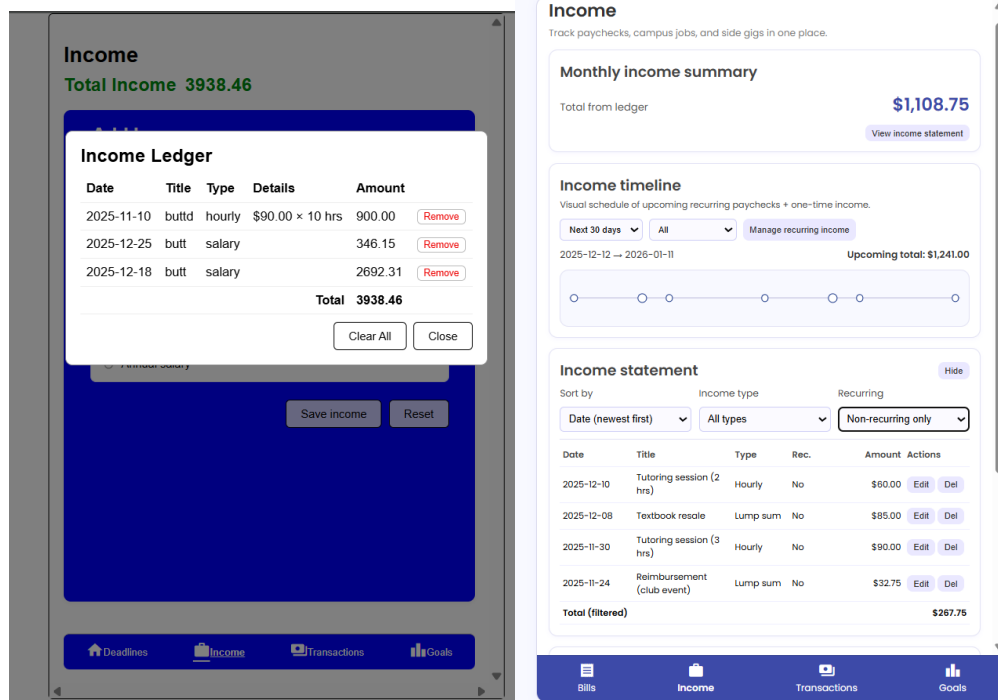
Title / name of income
e.g., Campus barista, Internship, tutoring

Type of income
Select type...

☐ Save as recurring income (for timeline)
Recurring works best with salary / stipend entries.

Reset Save income

Bills Income Transactions Goals



In the earlier version of the Income page, key information was not easy to understand at a glance. Users had to scan through entries and scroll to determine what income was coming in and when, which increased effort and reduced clarity. Recurring salaries and stipends were also not presented as a clear, editable option, making predictable income harder to manage consistently over time. In addition, the income statement needed a more usable structure for realistic, scalable data, with clearer sorting/filtering controls and straightforward edit/delete actions that would not make the screen feel crowded or require excessive scrolling.

After incorporating the Phase 2.1 feedback, we revised the page to improve readability, reduce scrolling, and make recurring income a prominent feature. We added an “Income Timeline” that visually summarizes upcoming recurring paychecks and one-time income with controls for time range and filtering so users can interpret upcoming income with minimal effort. We also added a “Manage recurring income” option that allows users to edit recurring fields like title, pay frequency, per-paycheck amount, and next pay date, making it easier to keep recurring entries accurate as schedules change. Finally, we refined the income statement so it remains structured and scalable, with sorting/filtering controls and consistent Edit/Delete actions.

Part B) AI Incorporation

Idea #1: Receipt Scanner

The Receipt Scanner feature would use AI-powered image recognition to simplify transaction entry within the budgeting app. By using the device camera or uploading a photo of a receipt, the AI would automatically extract transaction details, including name, amount, date, and category. These details would then automatically get added into the transaction history, reducing the need for manual input while allowing users to quickly verify or edit any information before finalizing the entry.

From a Human-AI Interaction perspective, the AI performs the complex task of reading and interpreting receipts, but the user retains control over confirming or correcting entries. Bias and errors may arise if the AI misreads uncommon receipt formats or handwriting, so regular training on different receipt types would help improve accuracy. This “human-in-the-loop” design also supports trust, as users can see the AI’s extracted data and make adjustments if necessary. Ethical considerations include handling of sensitive financial data properly and ensuring transparency about what the AI does, allowing users to feel both informed and in control.

Idea #2: Smart Budget Advisor

The Smart Budget Advisor would use ML tools to provide personalized financial guidance within the app. On the Goals page, where users manually enter spending or saving goals and record progress through transactions or savings, the AI would analyze old spending patterns and current goal progress to offer recommendations. For example, it could predict whether a user is likely to reach a particular savings target by a set date and suggest adjustments to daily, weekly, or monthly spending to stay on track.

From a Human-AI Interaction perspective, the system maintains user agency, so the AI only suggests adjustments, and users decide whether to act on them. Bias could arise if the model relies heavily on old spending habits, potentially encouraging unrealistic savings, so the system would need to be trained appropriately and provide clear explanations for each recommendation. A “human-in-the-loop” design ensures users remain in control of their goals while learning from AI insights. Trust and ethical considerations including transparency in how recommendations are generated, ensuring the AI does not push unsafe or stressful financial behaviors, and securely handling sensitive financial data are also very important.

Part C) Hypothetical “Phase 5” Timeline

Week 1: Prioritize the full implementation of the features conveyed in Phase 4. For example, having a paid bill on the “Bills” page actually become a transaction on the “Transactions” page, and maybe even show up in the spending in “Goals”. Update the style guide to reflect the new color palette and typography.

Week 2: Work on adding new frontend features that haven't been explored before. For example, a time navigation system with "Previous/Next Month" buttons and a dropdown for Year. This way, the user inputted information can be organized by time period.

Week 3: Conduct another user-testing session. Since there have been some dramatic changes since the Phase 2 prototype, it would make sense to receive additional feedback to make sure that all of the major and minor issues from before have been addressed. Ask the same testers as from Phase 3, but also recruit new users for fresh perspectives.

Week 4: Start preparing for backend integration. Do research on the technology stack for backend development. Begin implementing the initial structure. Set up a lightweight server framework such as Node.js with Express or Python Flask, and establish basic API routes for bills, income, transactions, and goals.

Week 5: Continue work on the backend. Connect the frontend to the endpoints and ensure that time-based filtering works consistently. Connect the backend to a database (such as PostgreSQL) so user data can be stored permanently and saved across sessions.

Week 6: Begin working on the user profile system. Create a "Profile" page where users can view and edit personal information like their name, email, and currency. Integrate backend support by assigning each dataset to a specific user ID. Implement basic authentication.

Week 7: Conduct a final round of user testing. Utilize a variety of testing methods, including simple observation, interviews, and a combination of both. Ask previous testers, but also recruit new ones. Try to get a diverse set of testers from various demographic groups. Use the "Likes, Dislikes, and Design Ideas" technique internally to evaluate the prototype and get insight from the design team itself.

Week 8: Implement the final set of adjustments and polish the high-fidelity prototype.

Part D) Assumptions Inventory and Audience Reflection

Assumptions Inventory

1. Basic financial literacy
 - a. Users can understand concepts such as income, expenses, budgets, savings goals, and deadlines
 - b. Users know the difference between income types (salary, hourly, freelance)

- c. Users are motivated to set savings and spending limits and review progress over time
- 2. Comfort with digital application interfaces
 - a. Users can interpret progress bars, pie charts, and ledger summaries without internal guidance
 - b. Users can enter and edit data, navigate drop-down menus and sorting tools
 - c. Users can navigate between multiple tabs
- 3. Access to compatible device
 - a. Users have consistent access to a device with a screen that fits our application (e.g. tablet)
- 4. Accessibility
 - a. Users have no significant visual impairments affecting color perception (progress bar changes, red urgency cues in deadlines)
 - b. Users can read English and understand numeric indicators

Audience Reflection

Keeping the aforementioned assumptions in mind, there are definitely a couple potential users that may not be well-served by our application. Specifically, individuals with limited financial literacy, or who are new to budgeting, may not be able to use the app effectively without additional guidance or tutorials. Individuals with accessibility barriers, and specifically those with visual impairments, may not be able to fully interpret our color-coding for deadlines, color-based progress bars, and charts. Individuals with complex financial structures (i.e. multiple irregular income streams) may find our app to be too simple for their tracking needs. Additionally, users who prefer automated data syncing through a bank account connection may also find our manual-entry system to be tedious and inefficient. Finally, users who require multilingual support may have difficulty using our app, as our app is currently equipped to only support English. Overall, these limitations help to clarify which specific user groups are most aligned with our app's current scope, and highlight future opportunities for improvement.